

07 — Attention & Transformers

Time: ~6-7 hours | **Level:** Advanced

What you'll learn:

- Attention mechanism: letting the model "look back" at all positions
- Scaled dot-product attention (from scratch)
- Multi-head attention: multiple parallel attention patterns
- Positional encoding: giving the model a sense of order
- The full Transformer architecture
- Encoder-only (BERT) vs Decoder-only (GPT) vs Encoder-Decoder (T5)

Prerequisites: Notebooks 03-06 (neural networks, PyTorch, NLP, RNNs)

The Paper That Changed Everything

"Attention Is All You Need" (Vaswani et al., 2017) — the most cited ML paper ever. It introduced the Transformer architecture and made RNNs/LSTMs obsolete for most NLP tasks.

GPT, BERT, T5, LLaMA, Phi-3 — all are Transformers.

```
In [ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import math

torch.manual_seed(42)
```

1. Attention — The Core Idea

Problem (from Notebook 06): Seq2Seq squashes ALL input into ONE vector.

Solution: Let the decoder look at EVERY encoder position and decide which ones are relevant.

Intuition

You're reading: *"Patient was diagnosed with anxiety disorder. Prescribed sertraline 50mg."*

When generating a summary, for the word "medication", you should focus (attend) on "sertraline 50mg" — not "was diagnosed".

Query-Key-Value Framework

Attention uses three concepts (like a database lookup):

- **Query (Q)**: What am I looking for? (current decoder state)
- **Key (K)**: What do I contain? (each encoder position's identifier)
- **Value (V)**: What information do I have? (each encoder position's content)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Let's break this down step by step.

```
In [ ]: # — Scaled Dot-Product Attention from Scratch —————

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    THE fundamental operation of all Transformers.

    Args:
        Q: Queries (batch, seq_len_q, d_k)
        K: Keys (batch, seq_len_k, d_k)
        V: Values (batch, seq_len_k, d_v)
        mask: optional mask to prevent attending to certain positions

    Returns:
        output: weighted sum of values (batch, seq_len_q, d_v)
        attention_weights: (batch, seq_len_q, seq_len_k)
    """
    d_k = Q.shape[-1]

    # Step 1: Compute attention scores (how similar is each Q to each K?)
    scores = torch.matmul(Q, K.transpose(-2, -1)) # (batch, seq_q, seq_k)

    # Step 2: Scale by sqrt(d_k) to prevent softmax saturation
    # Without scaling, dot products grow with dimension → softmax becomes overconfident
    scores = scores / math.sqrt(d_k)

    # Step 3: Apply mask (optional – used in decoder to prevent looking ahead)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    # Step 4: Softmax → attention weights (probabilities that sum to 1)
    attention_weights = F.softmax(scores, dim=-1)

    # Step 5: Weighted sum of Values
    output = torch.matmul(attention_weights, V) # (batch, seq_q, d_v)

    return output, attention_weights
```

```
# — Demo —————
batch_size, seq_len, d_model = 1, 6, 8

# Imagine Q, K, V come from a 6-word sentence
Q = torch.randn(batch_size, seq_len, d_model)
K = torch.randn(batch_size, seq_len, d_model)
V = torch.randn(batch_size, seq_len, d_model)

output, weights = scaled_dot_product_attention(Q, K, V)

print(f'Q, K, V shape: ({batch_size}, {seq_len}, {d_model})')
print(f'Output shape: {output.shape}') # same as Q
print(f'Attention weights shape: {weights.shape}') # (1, 6, 6)
print(f'\nAttention weights (each row sums to 1):')
print(weights[0].detach().numpy().round(3))
print(f'Row sums: {weights[0].sum(dim=-1).detach().numpy().round(3)}')
```

```
In [ ]: # — Visualise attention weights —————

words = ['Patient', 'reports', 'severe', 'anxiety', 'and', 'panic']

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(weights[0].detach().numpy(), cmap='Blues')
ax.set_xticks(range(len(words)))
ax.set_yticks(range(len(words)))
ax.set_xticklabels(words, rotation=45)
ax.set_yticklabels(words)
ax.set_xlabel('Key (attending TO)')
ax.set_ylabel('Query (attending FROM)')
ax.set_title('Attention Weights (random – will be learned during training)')
plt.colorbar(im)
plt.tight_layout()
plt.show()

print('💡 After training, you\'d see "anxiety" attending strongly to "severe"')
print('This is what makes Transformers powerful – they learn WHICH words')
```

2. Multi-Head Attention

A single attention head can only learn ONE type of relationship (e.g., "adjective modifies noun").

Multi-head attention runs h attention heads in parallel, each learning different patterns:

- Head 1 might learn syntactic relationships
- Head 2 might learn semantic similarity
- Head 3 might learn coreference ("he" → "patient")

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

```
In [ ]: class MultiHeadAttention(nn.Module):
        """
        Multi-Head Attention from scratch.
        This is the exact implementation inside every Transformer.

        Phi-3 uses: d_model=3072, n_heads=32 → d_k=96 per head
        GPT-3 uses: d_model=12288, n_heads=96 → d_k=128 per head
        """
        def __init__(self, d_model, n_heads):
            super().__init__()
            assert d_model % n_heads == 0, 'd_model must be divisible by n_heads'

            self.d_model = d_model
            self.n_heads = n_heads
            self.d_k = d_model // n_heads  # dimension per head

            # Linear projections: these ARE the learned parameters
            self.W_q = nn.Linear(d_model, d_model)  # projects Q to all heads
            self.W_k = nn.Linear(d_model, d_model)  # projects K to all heads
            self.W_v = nn.Linear(d_model, d_model)  # projects V to all heads
            self.W_o = nn.Linear(d_model, d_model)  # final output projection

        def forward(self, Q, K, V, mask=None):
            batch_size = Q.shape[0]

            # 1. Project Q, K, V through linear layers
            Q = self.W_q(Q)  # (batch, seq, d_model)
            K = self.W_k(K)
            V = self.W_v(V)

            # 2. Reshape to (batch, n_heads, seq, d_k) – split into heads
            Q = Q.view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
            K = K.view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
            V = V.view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)

            # 3. Scaled dot-product attention (per head, in parallel)
            scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
```

```

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
        attn_weights = F.softmax(scores, dim=-1)
        attn_output = torch.matmul(attn_weights, V) # (batch, n_heads, seq,

# 4. Concatenate heads
attn_output = attn_output.transpose(1, 2).contiguous().view(batch_si

# 5. Final linear projection
output = self.W_o(attn_output)

return output, attn_weights

# Demo
mha = MultiHeadAttention(d_model=64, n_heads=8)
x = torch.randn(2, 10, 64) # batch=2, seq=10, d_model=64
out, attn = mha(x, x, x) # self-attention: Q=K=V=x
print(f'Input: {x.shape}')
print(f'Output: {out.shape}')
print(f'Attention: {attn.shape}') # (2, 8, 10, 10) - 8 heads, each 10x10
print(f'\n💡 8 attention heads, each with its own 10x10 attention pattern.')
print(f'    Total params in MHA: {sum(p.numel() for p in mha.parameters()):,}')

```

3. Positional Encoding

Attention is **permutation-invariant** — "dog bites man" and "man bites dog" produce the same attention scores if word embeddings are identical.

We need to inject **position information**. The original Transformer uses sinusoidal encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Modern models (GPT, LLaMA, Phi-3) use **Rotary Position Embeddings (RoPE)** instead, which encode relative positions more effectively.

```

In [ ]: # — Sinusoidal Positional Encoding —————

def sinusoidal_positional_encoding(max_len, d_model):
    """Generate position encodings as described in "Attention Is All You Need"
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len).unsqueeze(1).float() # (max_len, 1)

    # Compute the division term
    div_term = torch.exp(torch.arange(0, d_model, 2).float() * -(math.log(10000) / 2))

    pe[:, 0::2] = torch.sin(position * div_term) # even indices
    pe[:, 1::2] = torch.cos(position * div_term) # odd indices

```

```

    return pe

pe = sinusoidal_positional_encoding(100, 64)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Heatmap of positional encodings
axes[0].imshow(pe[:50, :].numpy(), aspect='auto', cmap='RdBu')
axes[0].set_xlabel('Embedding Dimension')
axes[0].set_ylabel('Position')
axes[0].set_title('Positional Encodings (first 50 positions)')

# A few dimensions across positions
for dim in [0, 1, 4, 5, 16, 17]:
    axes[1].plot(pe[:50, dim].numpy(), label=f'dim {dim}', alpha=0.7)
axes[1].set_xlabel('Position')
axes[1].set_ylabel('Value')
axes[1].set_title('Individual Dimensions Over Positions')
axes[1].legend(fontsize=8)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

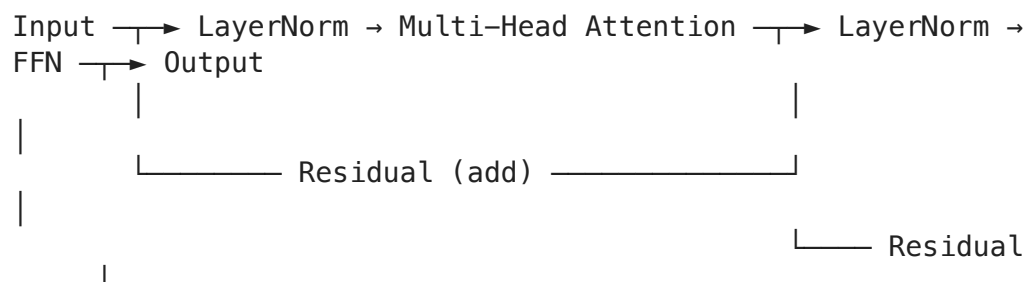
print('💡 Each position gets a UNIQUE pattern of sines and cosines.')
print('    Low-frequency dimensions capture long-range positions.')
print('    High-frequency dimensions capture fine-grained positions.')

```

4. Transformer Block

A single Transformer block combines:

1. Multi-Head Attention
2. Feed-Forward Network (2-layer MLP)
3. Layer Normalisation
4. Residual connections (skip connections)



Phi-3.5-mini stacks 32 of these blocks. GPT-3 uses 96.

```

In [ ]: class TransformerBlock(nn.Module):
        """

```

A single Transformer block – the building block of ALL modern LLMs.

Stack N of these = a Transformer model.

Phi-3: N=32, d_model=3072, n_heads=32

GPT-3: N=96, d_model=12288, n_heads=96

"""

```
def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
    super().__init__()
    # Multi-Head Attention
    self.attention = MultiHeadAttention(d_model, n_heads)
    self.norm1 = nn.LayerNorm(d_model)

    # Feed-Forward Network (expand then contract)
    self.ffn = nn.Sequential(
        nn.Linear(d_model, d_ff),      # expand (d_ff is usually 4 × d_model)
        nn.GELU(),                     # GELU activation (smoother than ReLU)
        nn.Linear(d_ff, d_model),      # contract back
    )
    self.norm2 = nn.LayerNorm(d_model)
    self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Self-attention with residual connection
        attn_out, _ = self.attention(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_out)) # residual + norm

        # FFN with residual connection
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_out)) # residual + norm

    return x

# Demo
block = TransformerBlock(d_model=64, n_heads=8, d_ff=256)
x = torch.randn(2, 10, 64) # batch=2, seq=10, d_model=64
out = block(x)
print(f'Input: {x.shape}')
print(f'Output: {out.shape}') # Same shape! → stackable
params = sum(p.numel() for p in block.parameters())
print(f'Parameters per block: {params:,}')
print(f'\n💡 Same input/output shape → blocks are stackable.')
print(f' 32 of these blocks = Phi-3 ({params * 32:,} params just in transf
```

5. Causal (Decoder) Masking

For **text generation** (GPT, LLaMA, Phi-3), the model must NOT see future tokens:

When predicting the 3rd word, it can only attend to words 1 and 2 — not words 4, 5, 6.

This is enforced with a **causal mask** (lower-triangular matrix):

```
In [ ]: # — Causal Mask —————
seq_len = 6
```

```

causal_mask = torch.tril(torch.ones(seq_len, seq_len)) # lower triangular

print('Causal mask (1 = can attend, 0 = blocked):')
print(causal_mask.int().numpy())

words = ['Patient', 'reports', 'severe', 'anxiety', 'and', 'panic']

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Without mask (encoder, BERT)
axes[0].imshow(torch.ones(6, 6).numpy(), cmap='Greens', vmin=0, vmax=1)
axes[0].set_title('Encoder (BERT): Full Attention\nEvery token sees every ot
axes[0].set_xticks(range(6))
axes[0].set_yticks(range(6))
axes[0].set_xticklabels(words, rotation=45, fontsize=8)
axes[0].set_yticklabels(words, fontsize=8)

# With causal mask (decoder, GPT)
axes[1].imshow(causal_mask.numpy(), cmap='Greens', vmin=0, vmax=1)
axes[1].set_title('Decoder (GPT): Causal Attention\nEach token only sees pas
axes[1].set_xticks(range(6))
axes[1].set_yticks(range(6))
axes[1].set_xticklabels(words, rotation=45, fontsize=8)
axes[1].set_yticklabels(words, fontsize=8)

plt.tight_layout()
plt.show()

print('💡 GPT / LLaMA / Phi-3 are DECODER-ONLY Transformers (causal mask).')
print('BERT is ENCODER-ONLY (full attention, good for understanding).')
print('T5 is ENCODER-DECODER (encoder has full attention, decoder has cau

```

6. Full Decoder-Only Transformer

This is the architecture of GPT, LLaMA, and Phi-3 — what you'll actually fine-tune.

```

In [ ]: class DecoderOnlyTransformer(nn.Module):
        """
        Simplified GPT-like model.

        This is structurally the SAME architecture as:
        - GPT-2 (2019): 12 layers, 768 dim, 12 heads
        - LLaMA-7B (2023): 32 layers, 4096 dim, 32 heads
        - Phi-3.5-mini (2024): 32 layers, 3072 dim, 32 heads

        The differences are in scale and training data, not architecture.
        """
        def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, max_seq
            super().__init__()
            self.d_model = d_model

            # Token + Position embeddings
            self.token_embedding = nn.Embedding(vocab_size, d_model)
            self.position_embedding = nn.Embedding(max_seq_len, d_model)

```



```

        self.dropout = nn.Dropout(dropout)

        # Stack of Transformer blocks
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads, d_ff, dropout)
            for _ in range(n_layers)
        ])

        self.norm = nn.LayerNorm(d_model)
        self.output_head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, input_ids):
        batch_size, seq_len = input_ids.shape

        # Create causal mask
        mask = torch.tril(torch.ones(seq_len, seq_len, device=input_ids.device))
        mask = mask.unsqueeze(0).unsqueeze(0) # (1, 1, seq, seq) for broadcast

        # Embeddings
        positions = torch.arange(seq_len, device=input_ids.device).unsqueeze(0)
        x = self.token_embedding(input_ids) + self.position_embedding(positions)
        x = self.dropout(x)

        # Pass through transformer blocks
        for block in self.blocks:
            x = block(x, mask)

        x = self.norm(x)
        logits = self.output_head(x) # (batch, seq, vocab_size)

        return logits

# Create a mini GPT
mini_gpt = DecoderOnlyTransformer(
    vocab_size=1000,
    d_model=128,
    n_heads=4,
    n_layers=4,
    d_ff=512,
    max_seq_len=64
)

total_params = sum(p.numel() for p in mini_gpt.parameters())
print(f'Mini GPT architecture:')
print(mini_gpt)
print(f'\nTotal parameters: {total_params:,}')
print(f'\nFor reference:')
print(f'  GPT-2:          124,000,000 params')
print(f'  Phi-3.5-mini: 3,800,000,000 params')
print(f'  GPT-3:          175,000,000,000 params')

```

```

In [ ]: # — Test generation (random – untrained) —————

def generate(model, start_tokens, max_new_tokens=20, temperature=1.0):
    """Autoregressive generation – same algorithm as GPT."""
    model.eval()

```

```

tokens = start_tokens.clone()

with torch.no_grad():
    for _ in range(max_new_tokens):
        # Get logits for the last position
        logits = model(tokens)[:,-1,:] # (batch, vocab_size)

        # Apply temperature (lower = more deterministic)
        logits = logits / temperature

        # Sample from probability distribution
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1) # (batch,

        # Append to sequence
        tokens = torch.cat([tokens, next_token], dim=1)

    return tokens

# Generate (random output since model is untrained)
input_ids = torch.tensor([[42, 100, 200]]) # 3 "seed" tokens
generated = generate(mini_gpt, input_ids, max_new_tokens=10)
print(f'Input tokens: {input_ids[0].tolist()}')
print(f'Generated: {generated[0].tolist()}')
print(f'\n💡 Output is random because weights are random.')
print('    Pre-training on internet text makes this output coherent.')
print('    Fine-tuning on our mental health data makes it domain-specific.')

```

7. Architecture Comparison

Architecture	Example	Attention	Best For
Encoder-only	BERT	Full (bidirectional)	Classification, NER, understanding
Decoder-only	GPT, LLaMA, Phi-3	Causal (left-to-right)	Text generation, chat, code
Encoder-Decoder	T5, BART	Encoder: full, Decoder: causal	Translation, summarisation

For our Mental Health project:

We use **Phi-3.5-mini** (decoder-only) because:

1. We want to **generate** clinical assessments (not just classify)
2. Decoder-only models are simpler to fine-tune
3. The instruction-tuned variant follows our prompt format naturally

Key Takeaways

1. **Attention** = learned weighted combination of all positions
2. **Scaled dot-product attention**: $\text{softmax}(QK^T / \sqrt{d_k})V$
3. **Multi-head attention**: multiple parallel attention patterns
4. **Positional encoding**: inject order information (sinusoidal or learned)
5. **Transformer block**: Attention + FFN + LayerNorm + Residual
6. **Causal masking**: prevents looking ahead (for generation)
7. Stack N transformer blocks = GPT / LLaMA / Phi-3

Next: 08 — [HuggingFace Ecosystem](#) — using pre-trained Transformers without building from scratch